SNOWFLAKE

CSV Data Loading & Validation Lab

End-to-end practical documentation with execution screenshots

Lab detail	Value
Platform	Snowflake Snowsight
Role used	SYSADMIN
Database	SNOWFLAKE_LAB_DB
Schema	RAW_DATA
Virtual warehouse	LAB_WH — SMALL
Source file	customers.csv
Destination table	CUSTOMERS
Execution date	01 November 2026

Outcome: 10 CSV records successfully parsed and loaded, verified, and lab resources cleaned up.

1. Objective and scope

The objective of this practical was to perform a complete CSV ingestion workflow in Snowflake. The exercise covered workspace preparation, database and schema creation, virtual warehouse configuration, target-table definition, CSV file-format configuration, internal-stage creation, file upload, staged-file inspection, COPY INTO ingestion, validation, object inspection, and cleanup.

2. Environment and object layout

Object	Name / configuration	Purpose
Database	SNOWFLAKE_LAB_DB	Container for the lab schema and data objects.
Schema	RAW_DATA	Organizes the target table, stage, and file format.
Warehouse	LAB_WH (SMALL)	Compute resource used to execute the SQL workload.
File format	CSV_FORMAT	Defines how CSV records and dates are parsed.
Internal stage	CSV_STAGE	Stores the uploaded customers.csv file.
Table	CUSTOMERS	Destination table for the ingested records.

The worksheet initially showed the SYSADMIN role and the account's default COMPUTE_WH warehouse. The database and schema were then created and selected. The lab warehouse was created with AUTO_SUSPEND = 60, AUTO_RESUME = TRUE, and INITIALLY_SUSPENDED = TRUE.

3. Workspace and session context

A SQL worksheet named snowflake_csv_ingestion_lab.sql was created in My Workspace. The session context was inspected using CURRENT_USER(), CURRENT_ROLE(), CURRENT_WAREHOUSE(), CURRENT_DATABASE(), and CURRENT_SCHEMA(). The initial result showed SYSADMIN and COMPUTE_WH, with no active database or schema selected.

```
SELECT
    CURRENT_USER() AS USER_NAME,
    CURRENT_ROLE() AS ACTIVE_ROLE,
    CURRENT_WAREHOUSE() AS ACTIVE_WAREHOUSE,
    CURRENT_DATABASE() AS ACTIVE_DATABASE,
    CURRENT_SCHEMA() AS ACTIVE_SCHEMA;
```

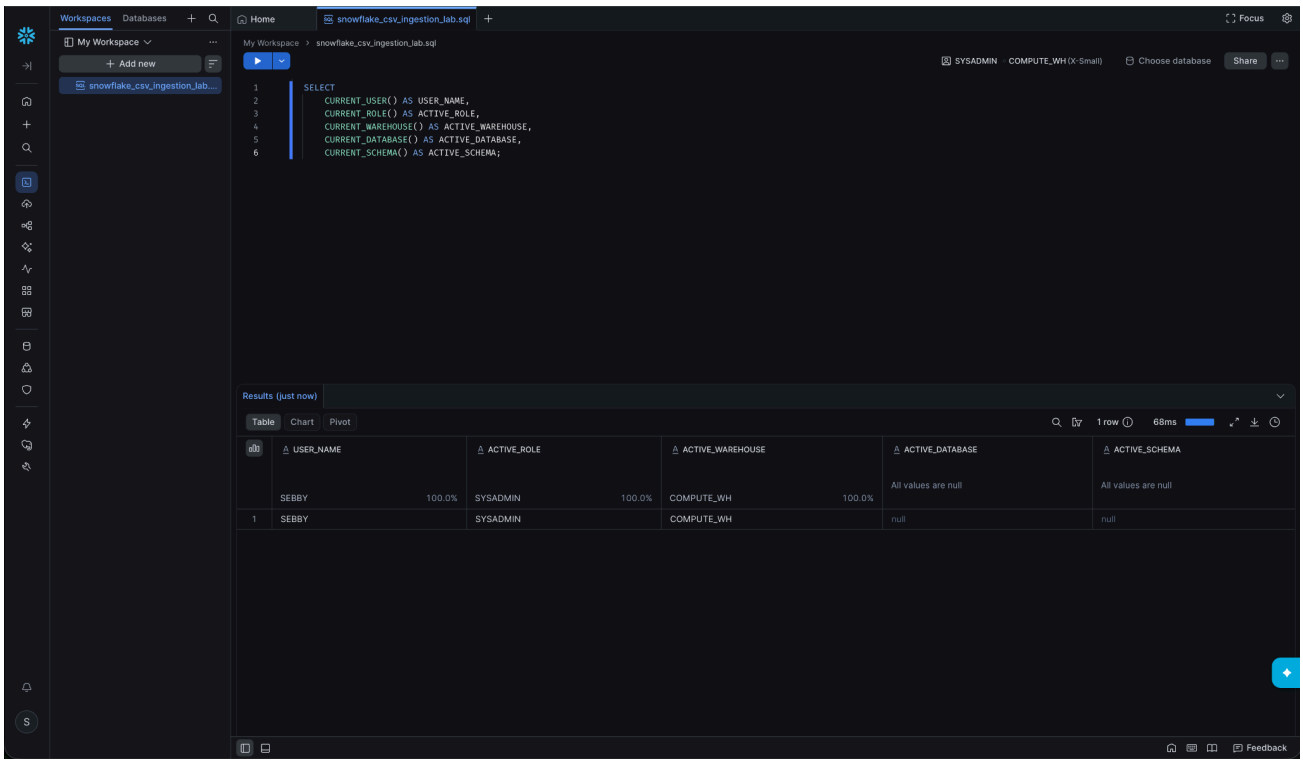


Figure 1. Initial worksheet and session context.

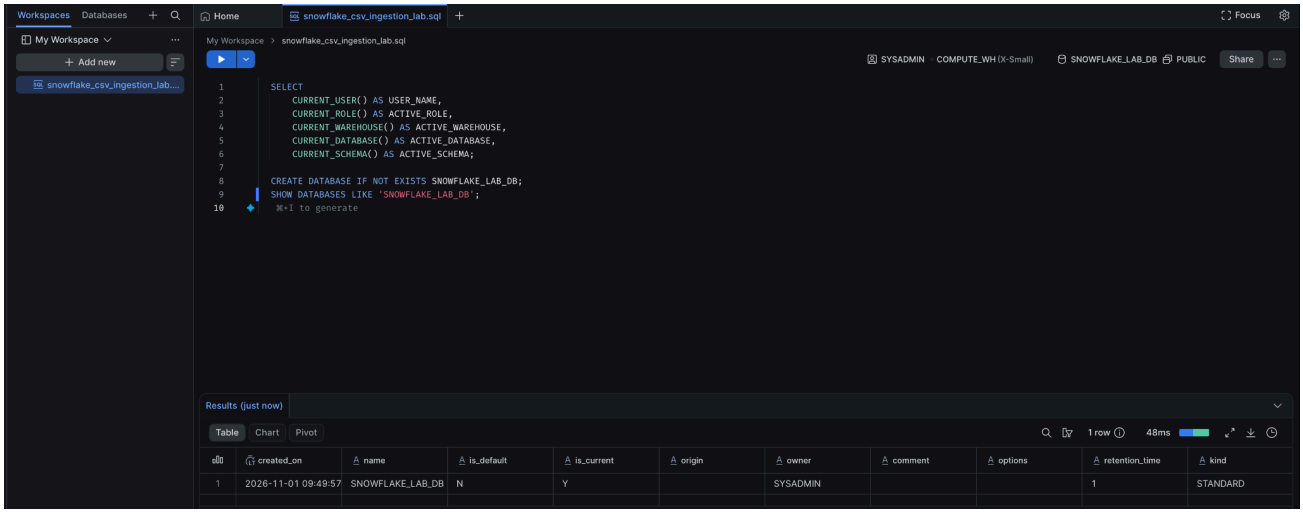


Figure 2. Database creation and SHOW DATABASES verification.

SNOWFLAKE_LAB_DB was created successfully. SHOW DATABASES returned the new database, owned by SYSADMIN.

4. Database and schema creation

```
CREATE DATABASE IF NOT EXISTS SNOWFLAKE_LAB_DB;
CREATE SCHEMA IF NOT EXISTS SNOWFLAKE_LAB_DB.RAW_DATA;

USE DATABASE SNOWFLAKE_LAB_DB;
USE SCHEMA RAW_DATA;
```

```
SHOW SCHEMAS IN DATABASE SNOWFLAKE_LAB_DB;
SELECT CURRENT_DATABASE(), CURRENT_SCHEMA();
```

The schema listing showed the built-in INFORMATION_SCHEMA and PUBLIC schemas as well as the newly created RAW_DATA schema. The session context query confirmed SNOWFLAKE_LAB_DB and RAW_DATA as the active database and schema.

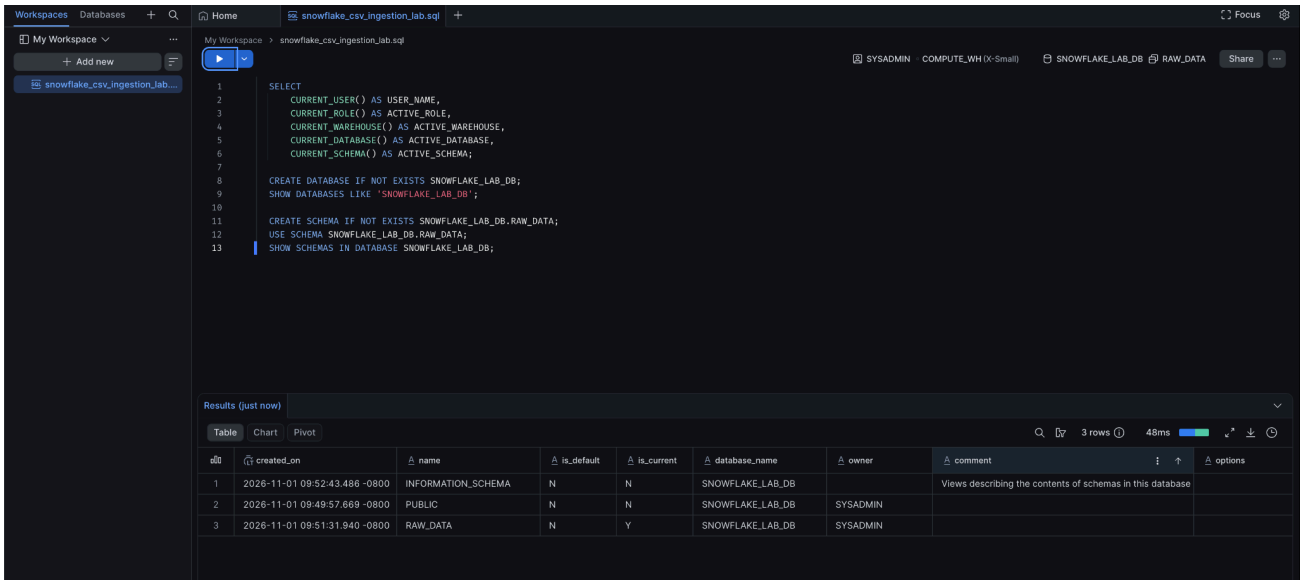


Figure 3. Creation and verification of RAW_DATA.

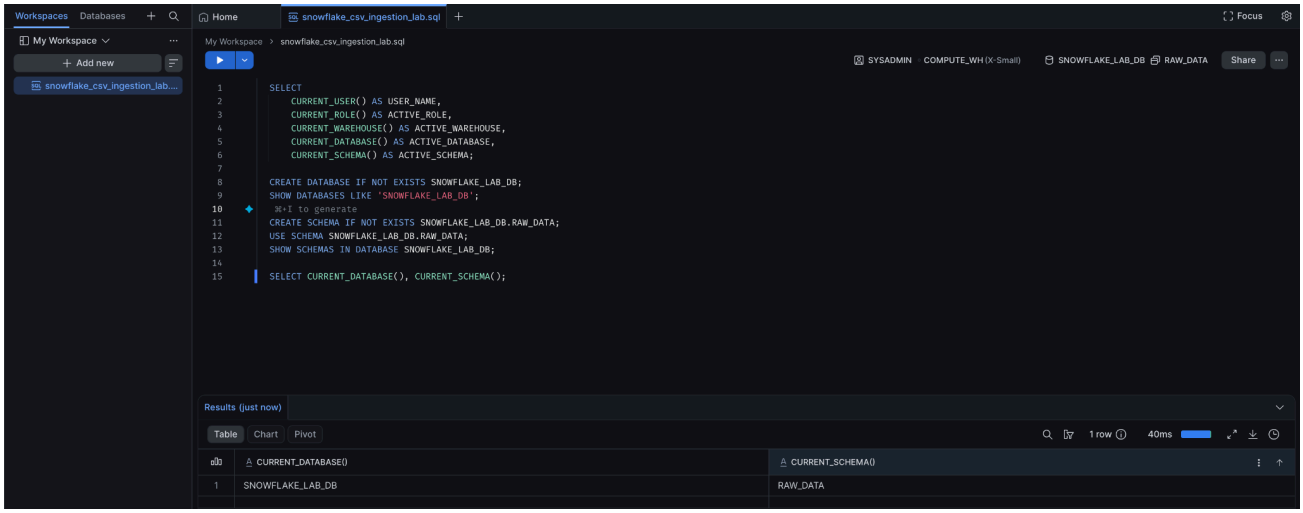


Figure 4. Session context confirms SNOWFLAKE_LAB_DB and RAW_DATA.

5. Virtual warehouse provisioning

```
CREATE WAREHOUSE LAB_WH
    WAREHOUSE_SIZE = 'SMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;

USE WAREHOUSE LAB_WH;
SHOW WAREHOUSES LIKE 'LAB_WH';
```

LAB_WH was created as a SMALL standard warehouse. Its initial state was SUSPENDED, which is consistent with INITIALLY_SUSPENDED = TRUE. AUTO_SUSPEND was set to 60 seconds and AUTO_RESUME was enabled. When subsequent SQL work required compute, the warehouse resumed and the later SHOW WAREHOUSES output showed its state as STARTED.

Parameter	Configured value	Meaning
WAREHOUSE_SIZE	SMALL	Compute size selected for this practical.
AUTO_SUSPEND	60	Suspend after 60 seconds of inactivity.
AUTO_RESUME	TRUE	Resume automatically when a query needs the warehouse.
INITIALLY_SUSPENDED	TRUE	Create the warehouse in a suspended state.

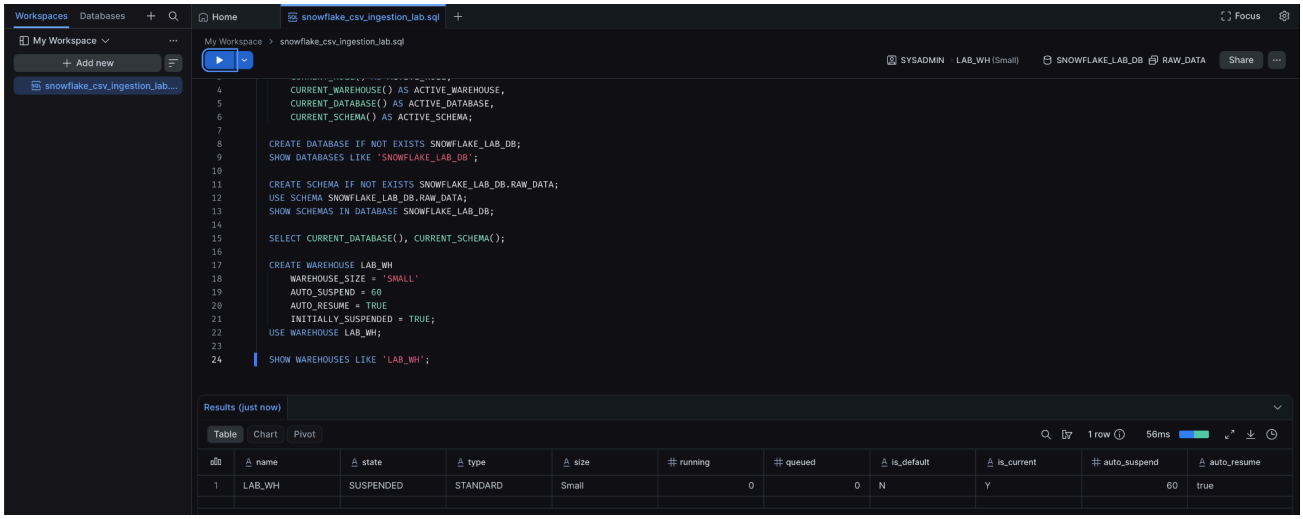


Figure 5. LAB_WH creation and initial SUSPENDED state.

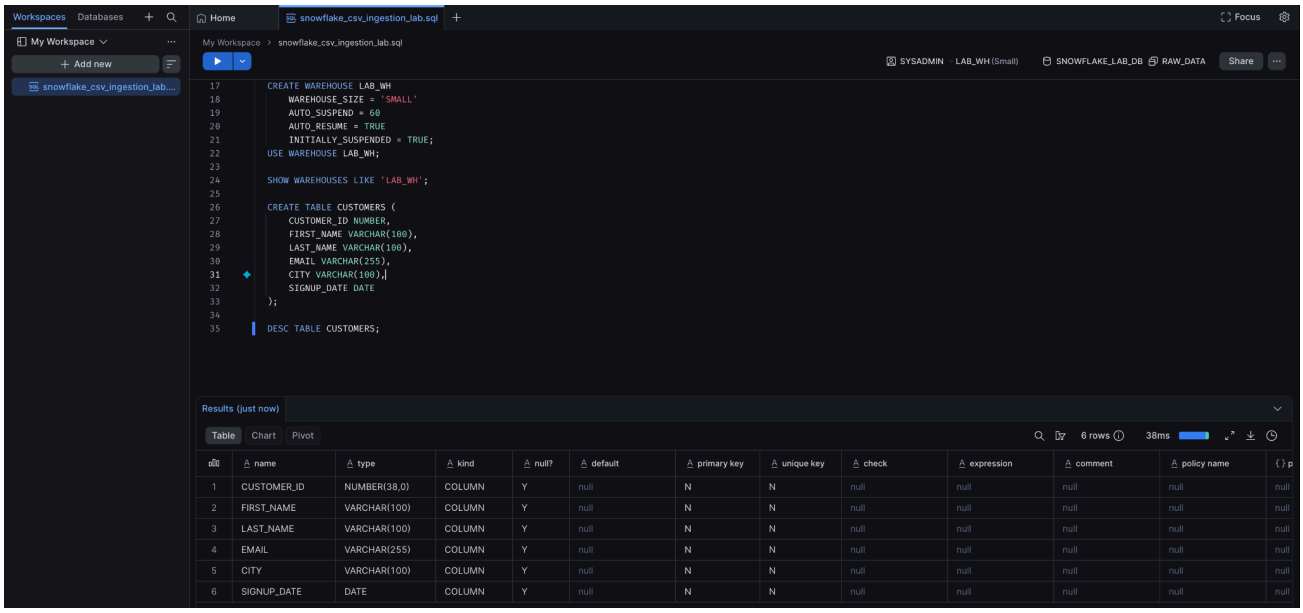


Figure 6. CUSTOMERS table creation and DESCRIBE TABLE output.

6. Destination table definition

The CUSTOMERS table was created in SNOWFLAKE_LAB_DB.RAW_DATA. DESCRIBE TABLE confirmed six columns and their Snowflake data types.

```
CREATE TABLE CUSTOMERS (
    CUSTOMER_ID NUMBER,
    FIRST_NAME VARCHAR(100),
    LAST_NAME VARCHAR(100),
    EMAIL VARCHAR(255),
    CITY VARCHAR(100),
    SIGNUP_DATE DATE
);

DESC TABLE CUSTOMERS;
```

Column	Data type	Description
CUSTOMER_ID	NUMBER(38,0)	Customer identifier
FIRST_NAME	VARCHAR(100)	Customer first name
LAST_NAME	VARCHAR(100)	Customer last name
EMAIL	VARCHAR(255)	Customer email address
CITY	VARCHAR(100)	Customer city
SIGNUP_DATE	DATE	Customer signup date

The DESCRIBE output showed all six columns as nullable. No primary-key or unique constraint was defined as part of this loading exercise.

7. CSV file format configuration

```
CREATE FILE FORMAT CSV_FORMAT
    TYPE = CSV
    FIELD_DELIMITER = ','
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    DATE_FORMAT = 'YYYY-MM-DD'
    EMPTY_FIELD_AS_NULL = TRUE;

SHOW FILE FORMATS LIKE 'CSV_FORMAT';
```

CSV_FORMAT was created in the RAW_DATA schema. It specifies comma-separated fields, one header row to skip, optional double-quote enclosure, YYYY-MM-DD date parsing, and conversion of empty fields to NULL.

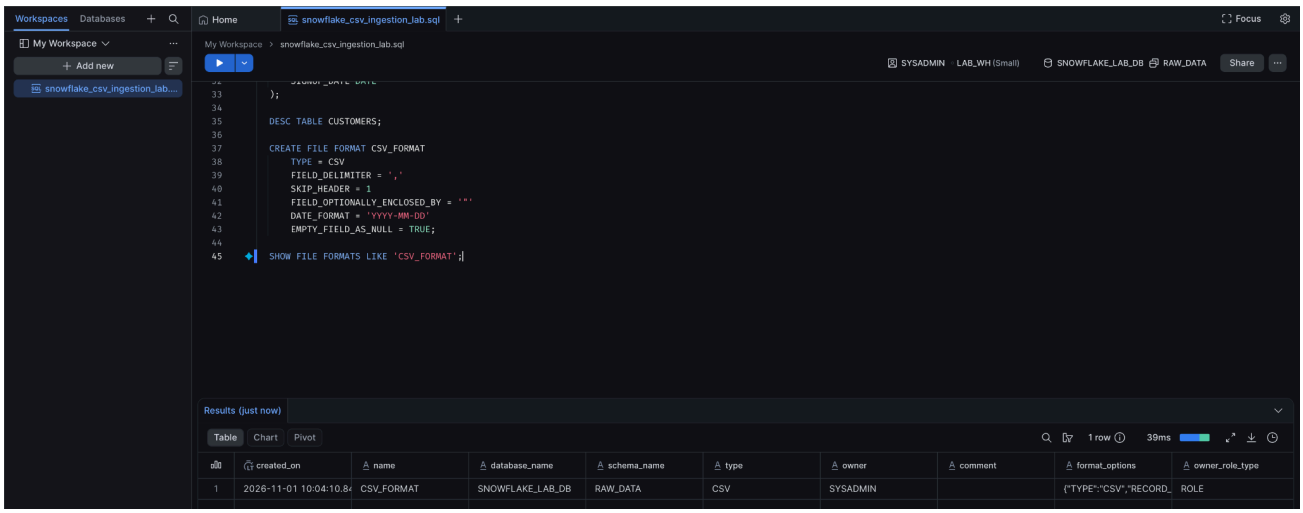


Figure 7. CSV_FORMAT creation and SHOW FILE FORMATS result.

8. Internal stage creation and inspection

```
CREATE STAGE CSV_STAGE
    FILE_FORMAT = CSV_FORMAT;

SHOW STAGES LIKE 'CSV_STAGE';
DESC STAGE CSV_STAGE;
```

CSV_STAGE was created as an internal named stage in the RAW_DATA schema. Its stage-level file-format property references CSV_FORMAT. DESC STAGE confirmed the configured STAGE_FILE_FORMAT / FORMAT_NAME value.

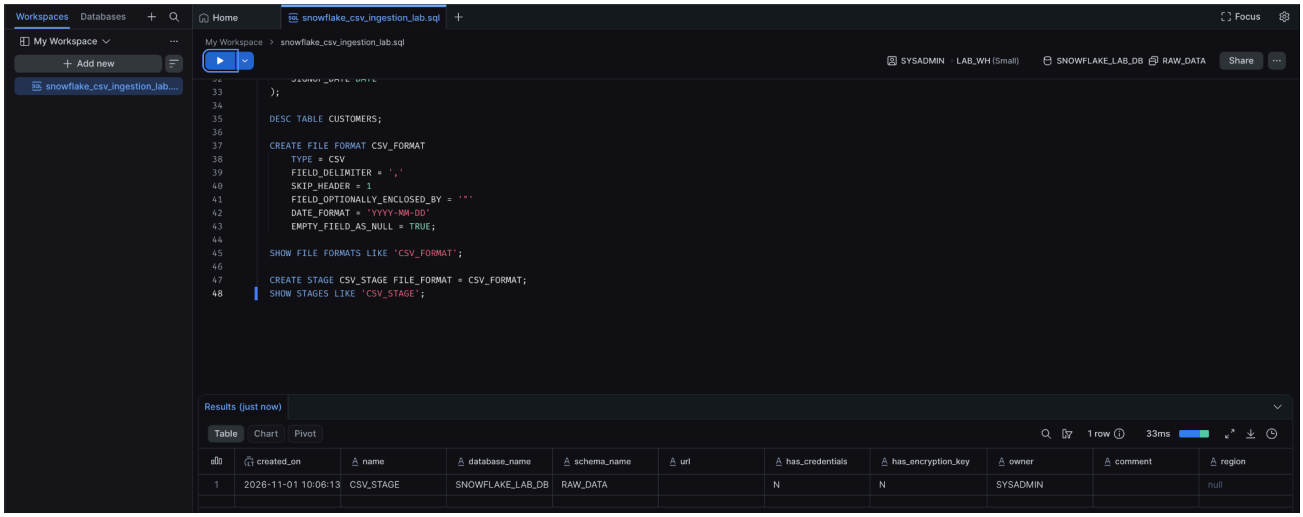


Figure 8. CSV_STAGE creation and SHOW STAGES verification.

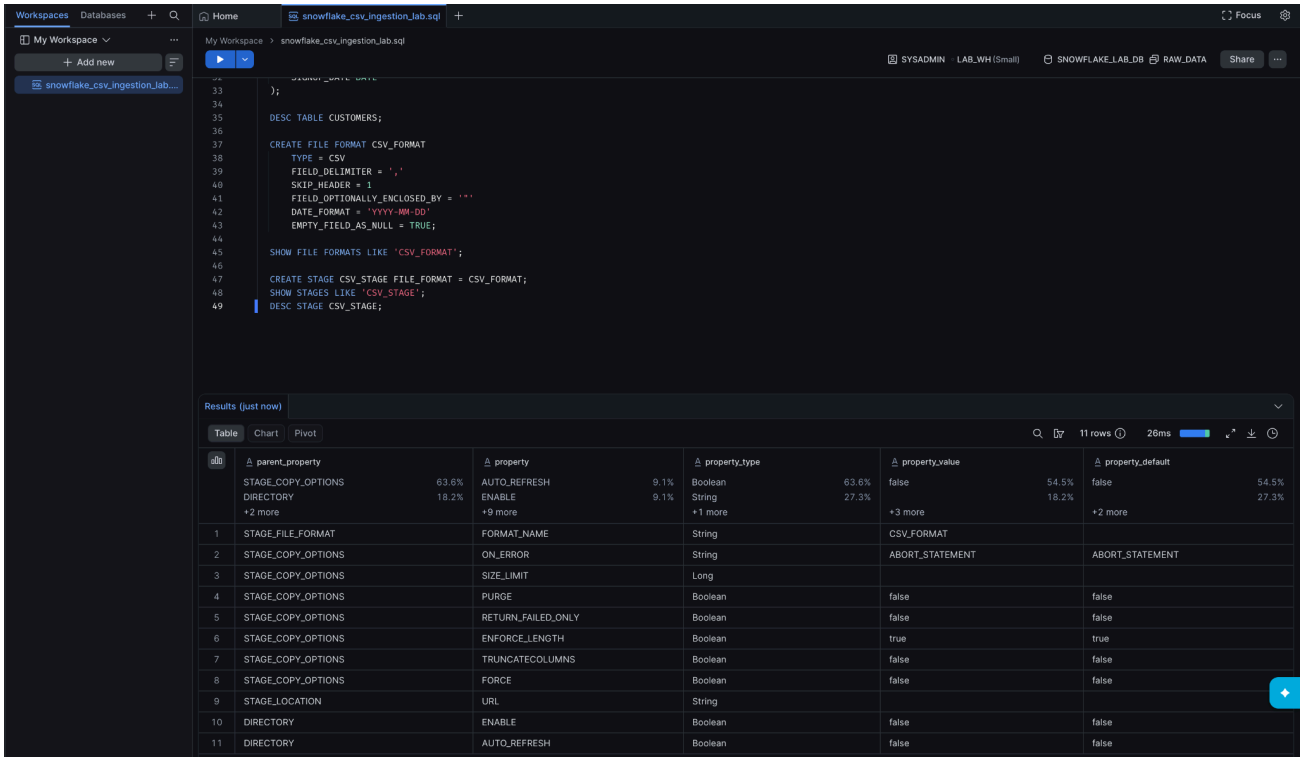


Figure 9. DESC STAGE confirms CSV_FORMAT association.

9. CSV upload and staged-file verification

The customers.csv file was uploaded to CSV_STAGE using the Snowsight stage upload workflow. The LIST command confirmed that the file was present. The returned object path was csv_stage/customers.csv, with a reported size of 608 bytes.

```
LIST @CSV_STAGE;
```

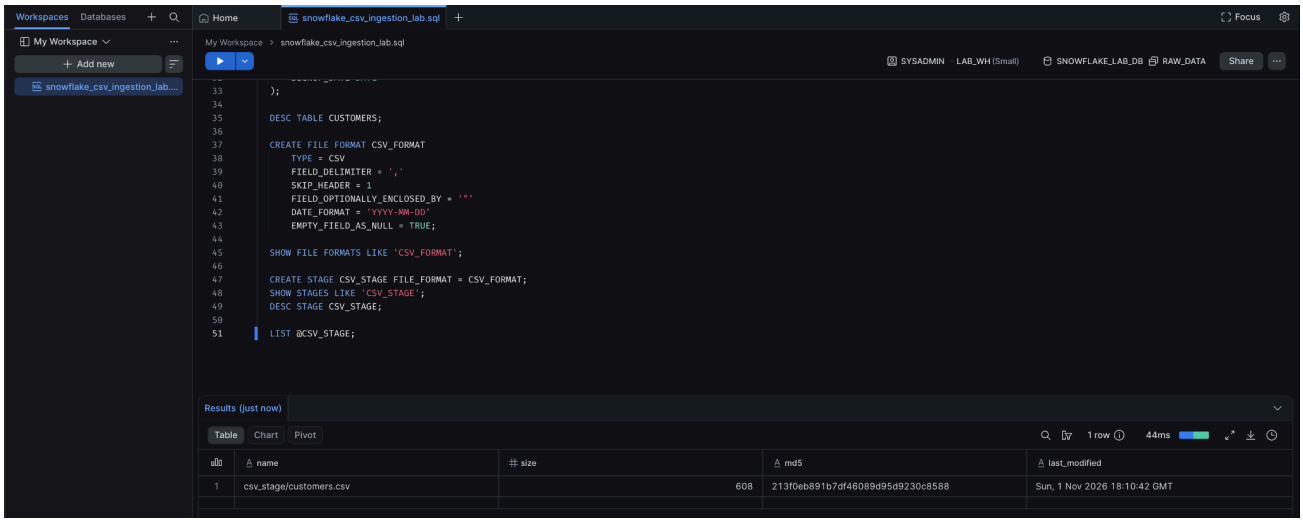


Figure 10. LIST @CSV_STAGE confirms customers.csv is staged.

10. Inspecting the staged CSV

Before loading the data into the destination table, a SELECT query read the staged file using CSV_FORMAT. Positional fields $1 through $6 were aliased to the expected customer column names. The result displayed ten data records, with the header excluded by SKIP_HEADER = 1.

```
SELECT
    $1 AS CUSTOMER_ID,
    $2 AS FIRST_NAME,
    $3 AS LAST_NAME,
    $4 AS EMAIL,
    $5 AS CITY,
    $6 AS SIGNUP_DATE
FROM @CSV_STAGE
(FILE_FORMAT => 'CSV_FORMAT');
```

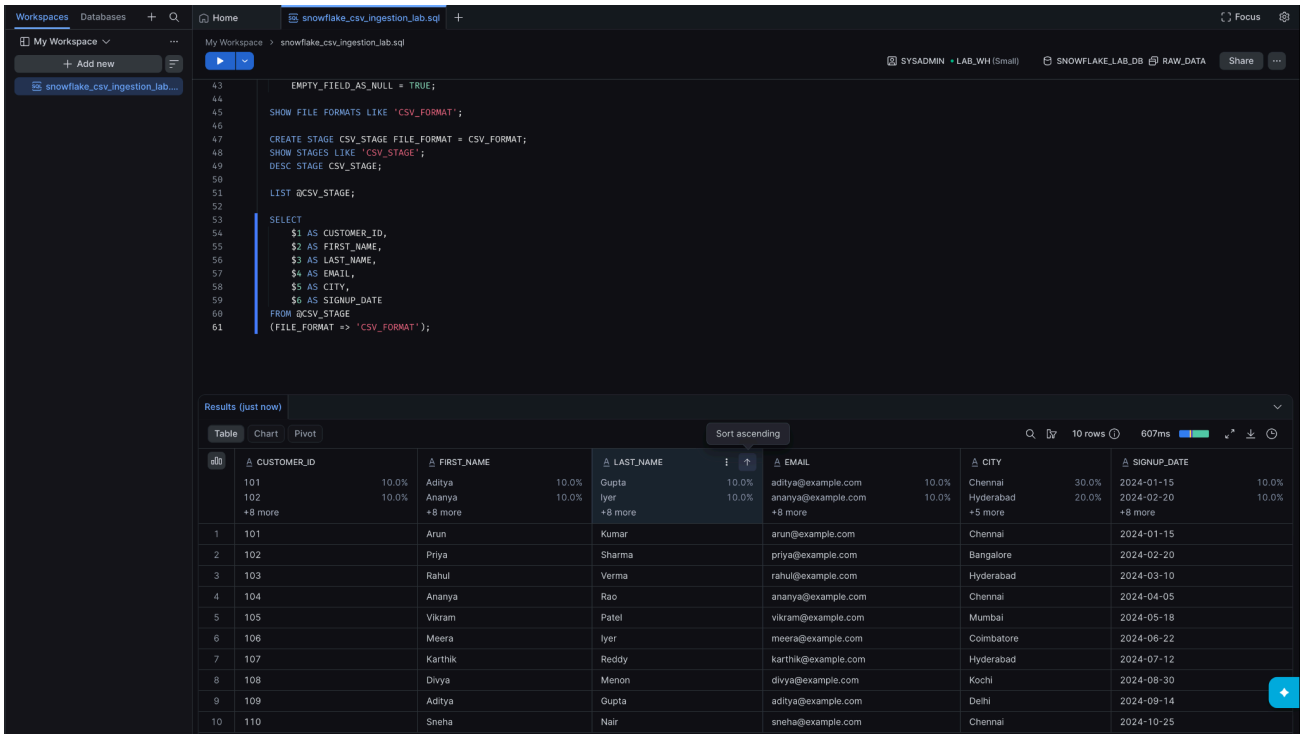


Figure 11. Staged-file preview returns ten parsed customer records.

11. Loading data with COPY INTO

COPY INTO loaded the staged CSV into CUSTOMERS. An explicit destination-column list was used. The SELECT statement read the six positional fields from the stage and cast them to the destination types. ON_ERROR = 'ABORT_STATEMENT' instructed Snowflake to fail the statement if a load error was encountered.

```
COPY INTO CUSTOMERS
(
    CUSTOMER_ID,
    FIRST_NAME,
    LAST_NAME,
    EMAIL,
    CITY,
    SIGNUP_DATE
)
```

```
FROM
(
    SELECT
        $1::NUMBER,
        $2::VARCHAR,
        $3::VARCHAR,
        $4::VARCHAR,
        $5::VARCHAR,
        $6::DATE
    FROM @CSV_STAGE
)
FILE_FORMAT = (FORMAT_NAME = 'CSV_FORMAT')
ON_ERROR = 'ABORT_STATEMENT';
```

The COPY INTO result reported status LOADED, 10 rows parsed, 10 rows loaded, and 0 errors. The first-error fields were NULL, indicating that no load error was reported.

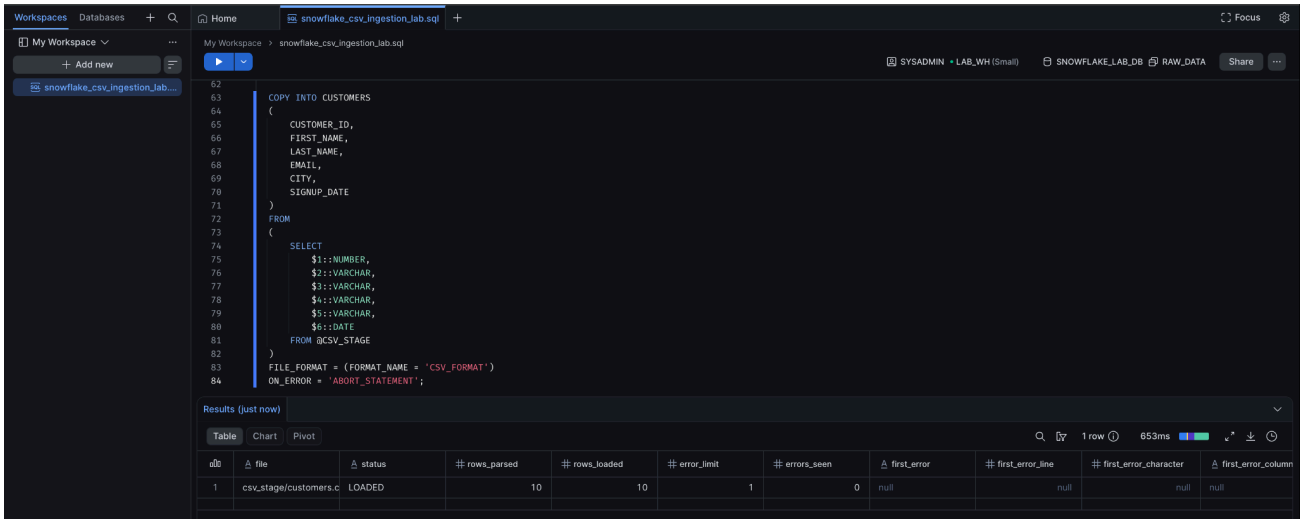


Figure 12. COPY INTO result: LOADED, 10 rows parsed, 10 rows loaded, zero errors.

12. Data validation

A COUNT(*) query returned 10 records. SELECT * FROM CUSTOMERS displayed all ten customer records, with the expected six columns and parsed signup dates.

```
SELECT COUNT(*) AS TOTAL_RECORDS
FROM CUSTOMERS;

SELECT *
FROM CUSTOMERS
ORDER BY CUSTOMER_ID;
```

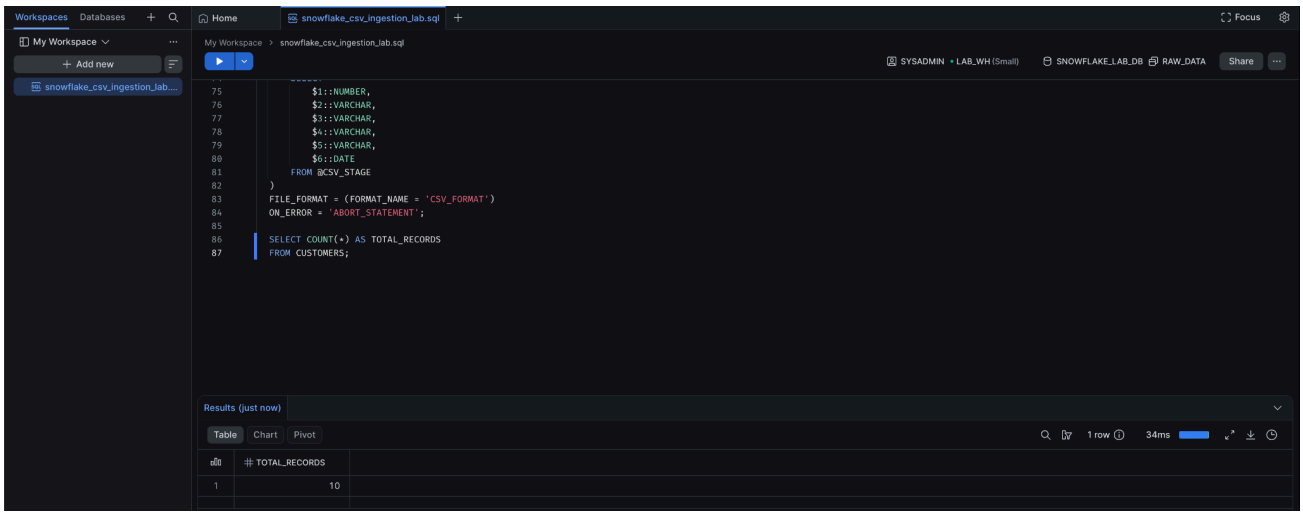


Figure 13. Record-count validation returns 10.

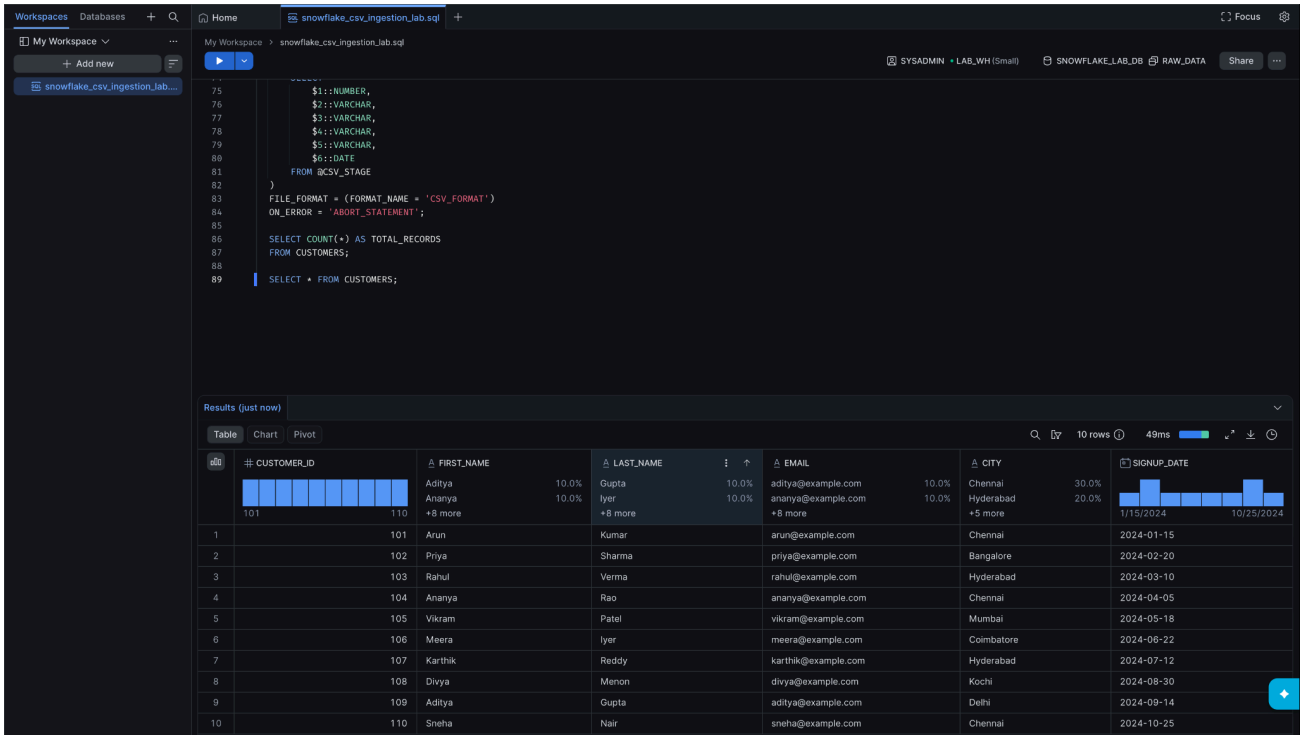


Figure 14. CUSTOMERS table displays the ten loaded records.

13. Snowflake object verification

```
SHOW DATABASES;
SHOW SCHEMAS IN DATABASE SNOWFLAKE_LAB_DB;
SHOW TABLES IN SCHEMA SNOWFLAKE_LAB_DB.RAW_DATA;
SHOW STAGES IN SCHEMA SNOWFLAKE_LAB_DB.RAW_DATA;
SHOW FILE FORMATS IN SCHEMA SNOWFLAKE_LAB_DB.RAW_DATA;
SHOW WAREHOUSES;
```

The object-verification commands were executed to inspect the database, schema, table, stage, file format, and warehouse. The warehouse listing showed LAB_WH in STARTED state at that point, because the workload had resumed it.

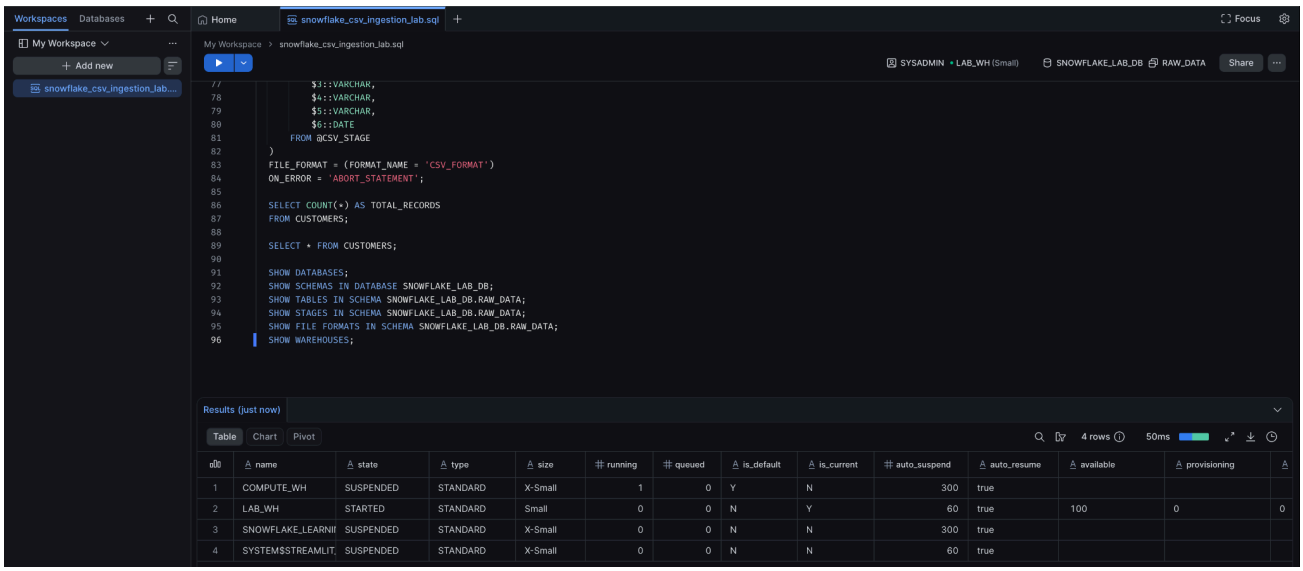


Figure 15. Object verification commands and warehouse listing.

14. Cleanup and resource removal

The staged CSV file was removed first. The table, stage, and file format were then dropped, followed by the RAW_DATA schema and SNOWFLAKE_LAB_DB database. Finally, LAB_WH was suspended and dropped. The final SHOW WAREHOUSES output no longer included LAB_WH.

```
REMOVE @CSV_STAGE;

DROP TABLE IF EXISTS CUSTOMERS;
DROP STAGE IF EXISTS CSV_STAGE;
DROP FILE FORMAT IF EXISTS CSV_FORMAT;

DROP SCHEMA IF EXISTS RAW_DATA;
DROP DATABASE IF EXISTS SNOWFLAKE_LAB_DB;

-- Suspend only if the warehouse is running.
ALTER WAREHOUSE LAB_WH SUSPEND;
DROP WAREHOUSE IF EXISTS LAB_WH;
```

Note: In this run, the staged file removal returned 'removed'. The final warehouse listing contained the account's other warehouses but no LAB_WH. The database and its child objects were removed by the DROP DATABASE operation.

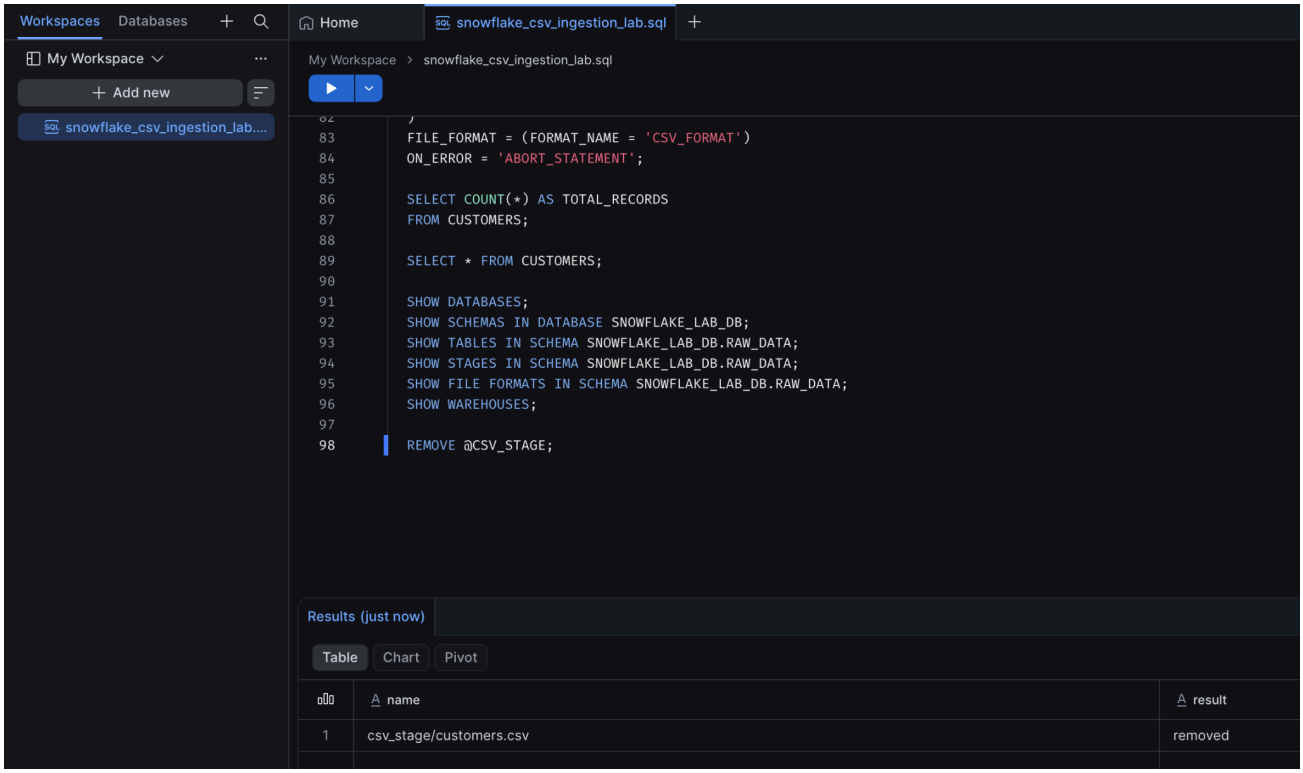


Figure 16. REMOVE @CSV_STAGE confirms the staged file was removed.

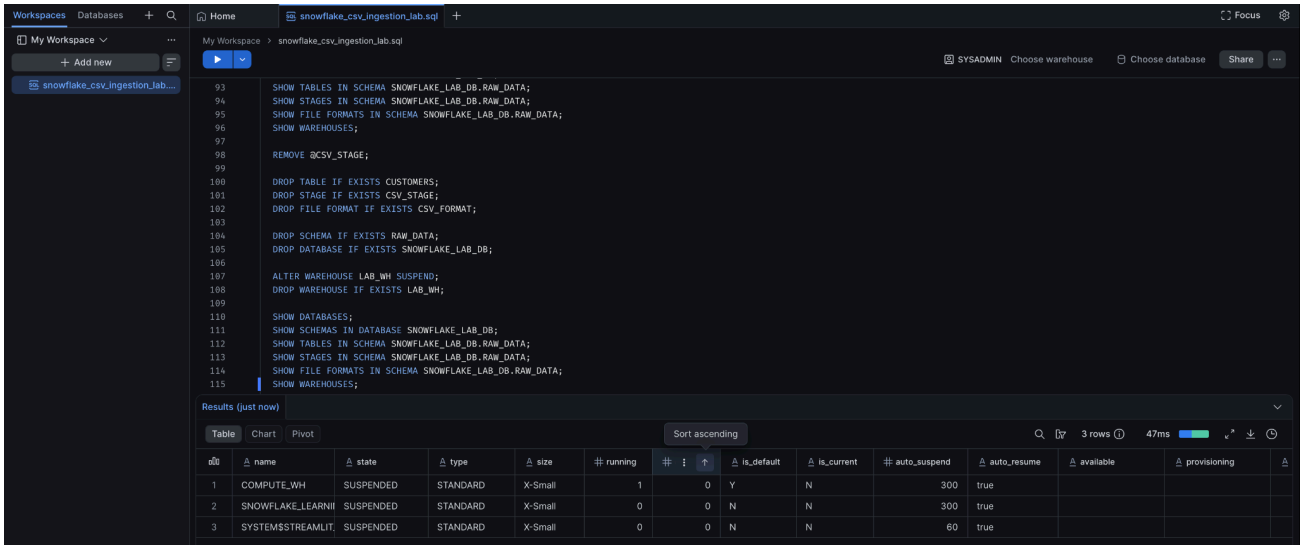


Figure 17. Final cleanup script and warehouse listing without LAB_WH.

15. Results and outcome

Validation	Observed result
Database and schema	SNOWFLAKE_LAB_DB and RAW_DATA created and selected
Warehouse	LAB_WH created as SMALL; auto-suspend 60 seconds; auto-resume enabled
Table definition	CUSTOMERS created with six columns
File format	CSV_FORMAT created and associated with CSV_STAGE

Stage upload	customers.csv listed in the internal stage
Staged-file preview	10 data rows returned
COPY INTO	LOADED; 10 rows parsed; 10 rows loaded; 0 errors
Table validation	COUNT(*) returned 10
Cleanup	Staged file, lab database objects, and LAB_WH removed

The practical successfully demonstrated an end-to-end Snowflake CSV ingestion workflow: provisioning Snowflake objects, configuring CSV parsing, uploading a file to an internal stage, previewing staged data, loading records with COPY INTO, validating the result, and cleaning up the lab resources.

16. Key concepts learned

- A database and schema organize Snowflake data objects; they do not provide query compute.
- A virtual warehouse supplies compute for SQL execution and can automatically suspend and resume.
- A file format defines how staged files are parsed; a stage stores files for loading.
- A staged file can be queried with positional references such as $1, $2, and $3.
- COPY INTO supports explicit destination-column mapping and typed transformations from staged data.
- The COPY INTO result provides file-level load status, parsed/loaded row counts, and error details.
- Warehouse suspension and warehouse deletion are different lifecycle operations.

Appendix: Final lab SQL script

The statements below summarize the main SQL workflow. The file upload itself was performed through Snowsight, not through this SQL script.

```
-- 1. Database and schema
CREATE DATABASE IF NOT EXISTS SNOWFLAKE_LAB_DB;
CREATE SCHEMA IF NOT EXISTS SNOWFLAKE_LAB_DB.RAW_DATA;
USE DATABASE SNOWFLAKE_LAB_DB;
USE SCHEMA RAW_DATA;

-- 2. Warehouse
CREATE WAREHOUSE LAB_WH
    WAREHOUSE_SIZE = 'SMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;
USE WAREHOUSE LAB_WH;

-- 3. Destination table
CREATE TABLE CUSTOMERS (
    CUSTOMER_ID NUMBER,
    FIRST_NAME VARCHAR(100),
    LAST_NAME VARCHAR(100),
    EMAIL VARCHAR(255),
    CITY VARCHAR(100),
    SIGNUP_DATE DATE
);
DESC TABLE CUSTOMERS;

-- 4. File format and stage
CREATE FILE FORMAT CSV_FORMAT
```

```
    TYPE = CSV
    FIELD_DELIMITER = ','
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    DATE_FORMAT = 'YYYY-MM-DD'
    EMPTY_FIELD_AS_NULL = TRUE;

CREATE STAGE CSV_STAGE FILE_FORMAT = CSV_FORMAT;
LIST @CSV_STAGE;

-- 5. Preview staged data
SELECT $1, $2, $3, $4, $5, $6
FROM @CSV_STAGE
(FILE_FORMAT => 'CSV_FORMAT');

-- 6. Load data
COPY INTO CUSTOMERS
(
    CUSTOMER_ID, FIRST_NAME, LAST_NAME,
    EMAIL, CITY, SIGNUP_DATE
)
FROM (
    SELECT
        $1::NUMBER, $2::VARCHAR, $3::VARCHAR,
        $4::VARCHAR, $5::VARCHAR, $6::DATE
    FROM @CSV_STAGE
)
FILE_FORMAT = (FORMAT_NAME = 'CSV_FORMAT')
ON_ERROR = 'ABORT_STATEMENT';

-- 7. Validate
SELECT COUNT(*) FROM CUSTOMERS;
SELECT * FROM CUSTOMERS ORDER BY CUSTOMER_ID;

-- 8. Cleanup
REMOVE @CSV_STAGE;
DROP DATABASE IF EXISTS SNOWFLAKE_LAB_DB;
DROP WAREHOUSE IF EXISTS LAB_WH;
```